

Student Attendance Record System

Project Report

Student Name	Aashika Jat
Program	B.Tech Computer Science & Engineering
Institution	VIT Bhopal University
Subject / Course	AI/ML
Submission Type	Flipped Course Evaluation — Build Your Own Project
Repository	https://github.com/aashikajat09/Attendance-Record-Section.git

1. Introduction

Attendance tracking in schools and colleges is often still done using paper registers or basic spreadsheets. This makes it slow to calculate a student's attendance percentage, tedious to identify students falling below an attendance threshold, and easy to lose or misplace records over a term.

This project implements a **Student Attendance Record System** as a menu-driven Python console application. It allows a teacher or administrator to register students, mark daily attendance, view attendance history, search individual records, calculate attendance percentages, flag students with low attendance, view a same-day summary, and persist all records to a text file at the end of a session.

The project demonstrates practical use of core Python constructs — lists, dictionaries, functions, loops, conditionals, string formatting, basic file handling, and input validation — combined into a single working application.

2. Problem Statement

Manual attendance tracking creates three recurring problems:

1. **Slow calculations** — working out a student's attendance percentage by hand, across many days, is time-consuming and error-prone.
2. **Difficult reporting** — identifying which students have fallen below a required attendance threshold means manually scanning every student's record.
3. **No persistence** — paper registers can be lost, damaged, or hard to search back through.

The system addresses these by keeping a single, consistent in-memory record for each student's present/absent counts, automatically calculating percentages on demand, and saving a plain-text copy of all records to disk before the program exits.

3. Functional Requirements

The system is organized into three major functional modules:

Module 1 — Student Management

- Register a new student with a unique roll number and name
- Prevent duplicate roll numbers and reject empty names
- Display all registered students with their present/absent counts

Module 2 — Attendance Marking & Records

- Mark a student Present or Absent by roll number, on demand
- Maintain running present/absent counts per student
- Track a same-session present/absent tally for a same-day summary
- Display a full attendance table with computed percentage per student

Module 3 — Search, Analysis & Reporting

- Search for a student by roll number and view their full record
- Calculate a single student's attendance percentage on request
- List all students whose attendance falls below a user-given threshold
- Show a same-day summary of total present/absent/overall attendance
- Persist all student records to `students.txt` on exit

Input / Output Structure

Action	Input	Output
Add student	Roll number, name	Confirmation or validation error
View all students	Menu choice	Table of all students with present/absent counts
Mark attendance	Roll number, P/A	Confirmation of status marked
View attendance records	Menu choice	Table with present, absent, and percentage per student
Search student	Roll number	Full student record with percentage
Calculate percentage	Roll number	That student's attendance percentage
Low attendance report	Threshold percentage	List of students below threshold
Today's summary	Menu choice	Present/absent/percentage for the session
Exit	Menu choice	Records written to <code>students.txt</code>

User Workflow

The application runs as a continuous loop: the main menu is displayed, the user selects an option (1–9), the corresponding module executes, a result is shown, and control returns to the menu until Exit is chosen, at which point records are saved to file. See Section 7.2 for the full workflow diagram.

4. Non-Functional Requirements

Requirement	How it is addressed
Usability	A clear, numbered menu (1–9) with descriptive labels lets a non-technical user operate the system without instructions. Feedback messages confirm every action (e.g. "Student X added successfully!").
Reliability	Attendance counts and percentages stay consistent across operations — <code>present / absent</code> fields are only ever updated through <code>mark_attendance()</code> , giving a single source of truth per student.
Error Handling / Robustness	Roll number input is validated with <code>.isdigit()</code> , empty names are rejected, duplicate roll numbers are blocked, invalid P/A input is caught, and division by zero is explicitly avoided in <code>get_percentage()</code> when a student has no attendance marked yet. File writing is wrapped in <code>try/except</code> .
Maintainability	The code is organized into small, single-purpose functions (<code>add_student()</code> , <code>mark_attendance()</code> , <code>low_attendance()</code> , etc.) with shared helper functions (<code>find_student()</code> , <code>get_percentage()</code>) to avoid duplicated logic.
Resource Efficiency	The system uses a single in-memory list of dictionaries with no external database, and only touches disk once (writing <code>students.txt</code> on exit), so it runs instantly on any machine with a standard Python interpreter.

5. System Architecture

The system follows a simple three-layer structure appropriate for a single-file, first-semester Python project: a presentation layer (the menu loop), a business logic layer (the feature functions), and a data layer (an in-memory list plus a text-file export on exit).

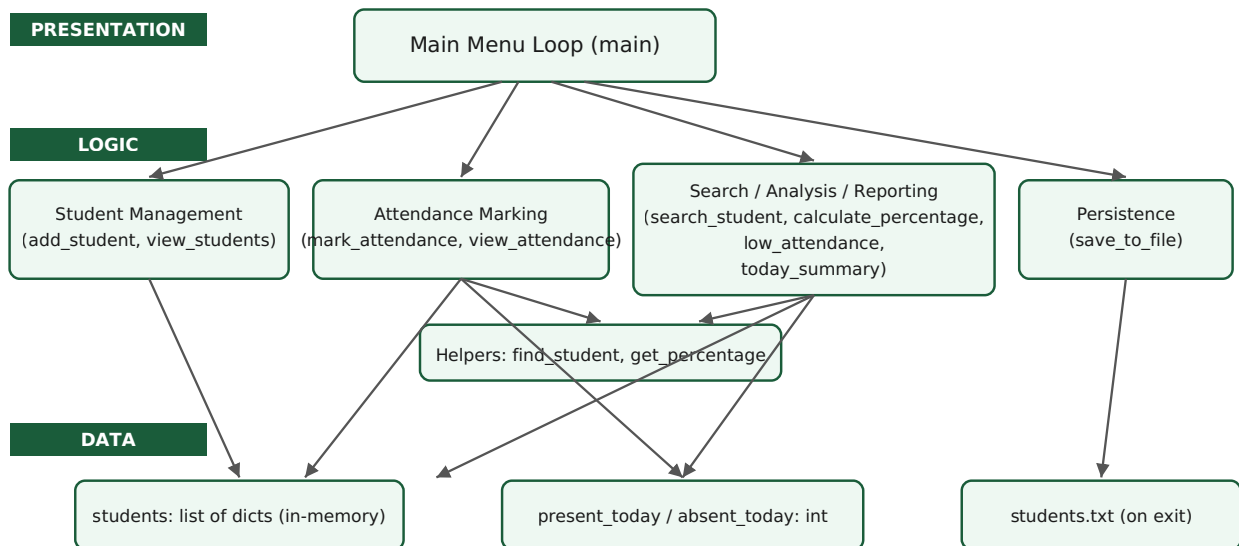


Figure 1: System Architecture Diagram

7. Design Diagrams

7.1 Use Case Diagram

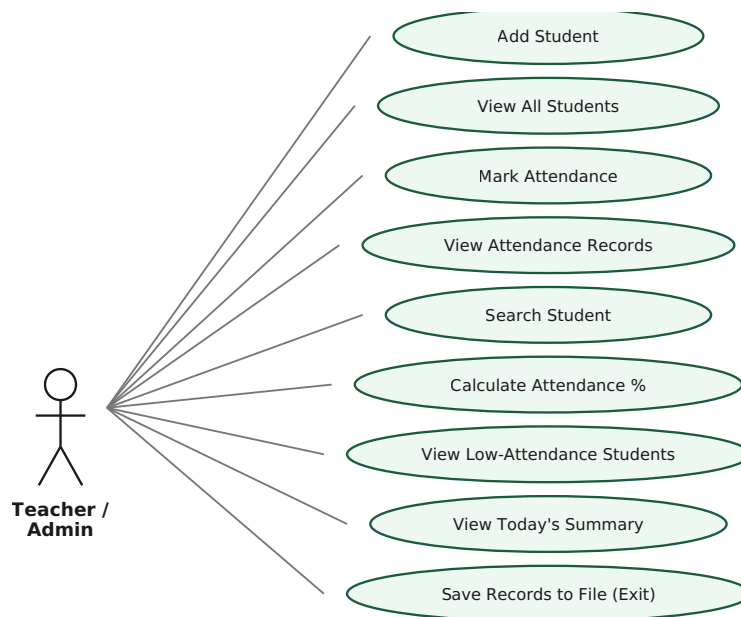


Figure 2: Use Case Diagram — Teacher / Administrator

7.2 Workflow Diagram

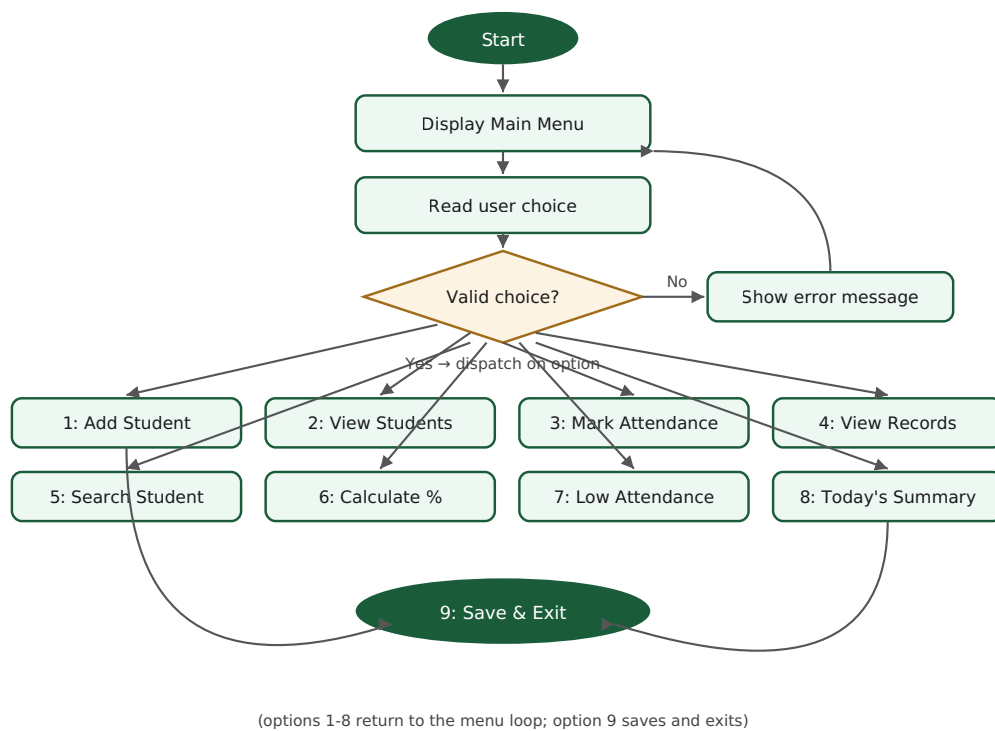


Figure 3: Main Menu Workflow

7.3 Class / Component Diagram

The implementation is function-based rather than class-based, consistent with first-semester scope. This diagram represents the system as functional components operating on one core record type.

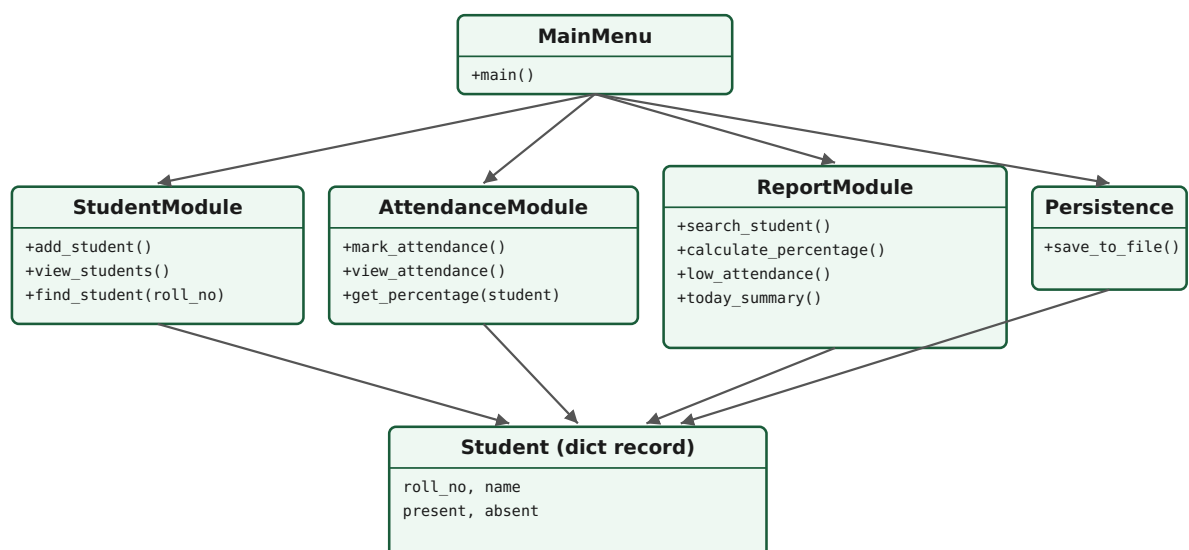


Figure 4: Component Diagram

7.4 Sequence Diagram — Marking Attendance

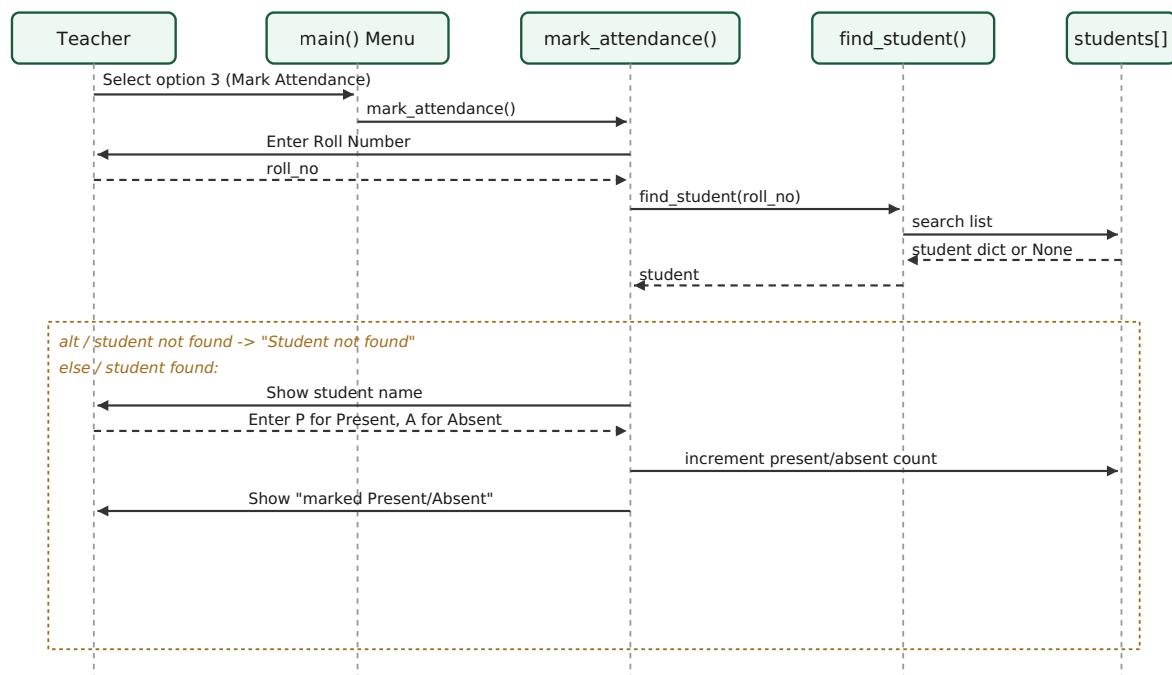


Figure 5: Sequence Diagram for the Mark Attendance flow

7.5 Storage / Schema Design

No external database is used during program execution — all data is held in-memory in a single list of dictionaries. On exit, this data is written out to a plain text file, `students.txt`, as a simple persistence mechanism.

students record (in-memory)		
Field	Type	Description
roll_no	int	Unique roll number (primary key)
name	str	Student name
present	int	Total number of days marked present
absent	int	Total number of days marked absent

students.txt output format (one block per student):

```
Roll No: 101
Name: Aditi Sharma
Present: 18
Absent: 2
```

Two additional session-only counters, `present_today` and `absent_today`, are kept outside the student records purely to support the "Today's Attendance Summary" feature and are not persisted to file.

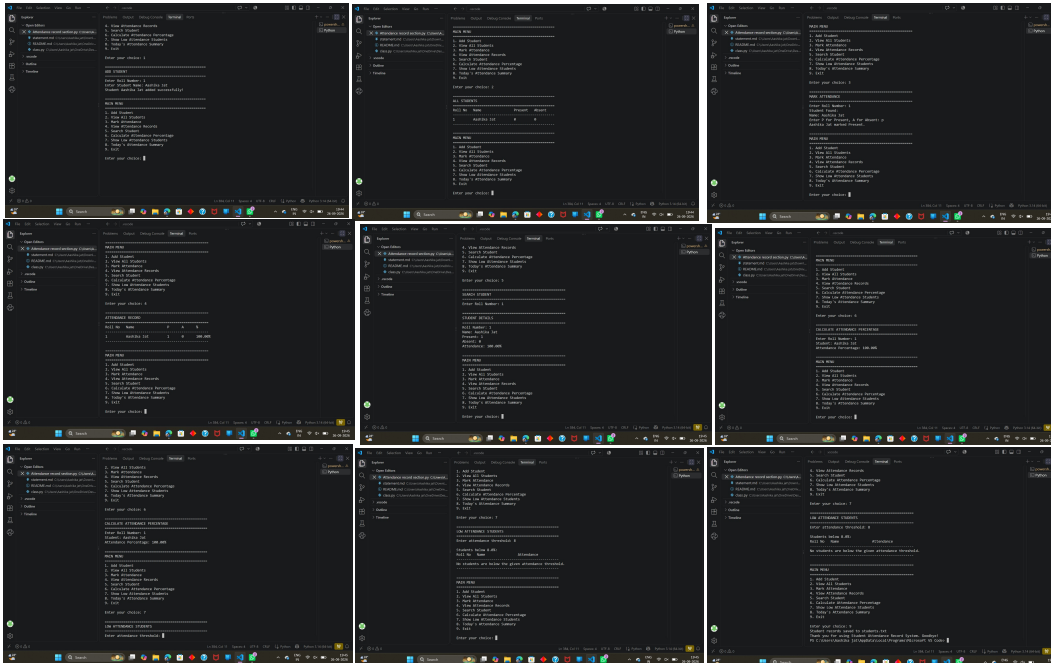
9. Implementation Details

The system is implemented in a single Python file using only the standard library — no external dependencies are required. Key implementation points:

- **Data storage:** `students` is a module-level list of dictionaries acting as an in-memory database for the program's runtime.
- **Lookup helper:** `find_student(roll_no)` centralizes the linear-search logic used by `mark_attendance()`, `search_student()`, and `calculate_percentage()`, avoiding duplicated loops.
- **Percentage helper:** `get_percentage(student)` computes `present / (present + absent) * 100` and explicitly guards against division by zero by returning `None` when no attendance has been recorded, with every caller checking for that case before formatting output.
- **Validation:** roll numbers are checked with `str.isdigit()` before conversion to `int`; names are checked for emptiness; duplicate roll numbers are blocked in `add_student()`; attendance-marking input is restricted to `P / A`.
- **File handling:** `save_to_file()` opens `students.txt` in write mode, writes each student's record as a labeled text block, and wraps the operation in `try/except` to catch any I/O error.

- **Menu loop:** `main()` runs a `while True` loop reading a menu choice and dispatching to the relevant function via `if/elif`, saving to file and exiting cleanly on option 9.

10. Screenshots / Results



14. Future Enhancements

Possible directions to extend this project beyond first-semester scope:

- Store attendance as a dated log (per-day records) instead of running totals, to support querying attendance on a specific past date
- Persist and reload data using JSON or CSV instead of plain text, so records survive between runs and can be re-imported
- Migrate storage to a database (SQLite) for larger class sizes
- Add a graphical interface (Tkinter) instead of a command-line menu
- Support multiple classes/sections rather than one flat student list
- Add authentication so only authorized staff can mark or edit attendance

15. References

- Python Software Foundation, *The Python Standard Library* — <https://docs.python.org/3/library/>
- Python Software Foundation, *Python Tutorial — Data Structures* — <https://docs.python.org/3/tutorial/datastructures.html>
- Python Software Foundation, *Reading and Writing Files* — <https://docs.python.org/3/tutorial/inputoutput.html#reading-and-writing-files>

- [Add any course material, textbook, or additional resource you referred to]